Theory of Computation

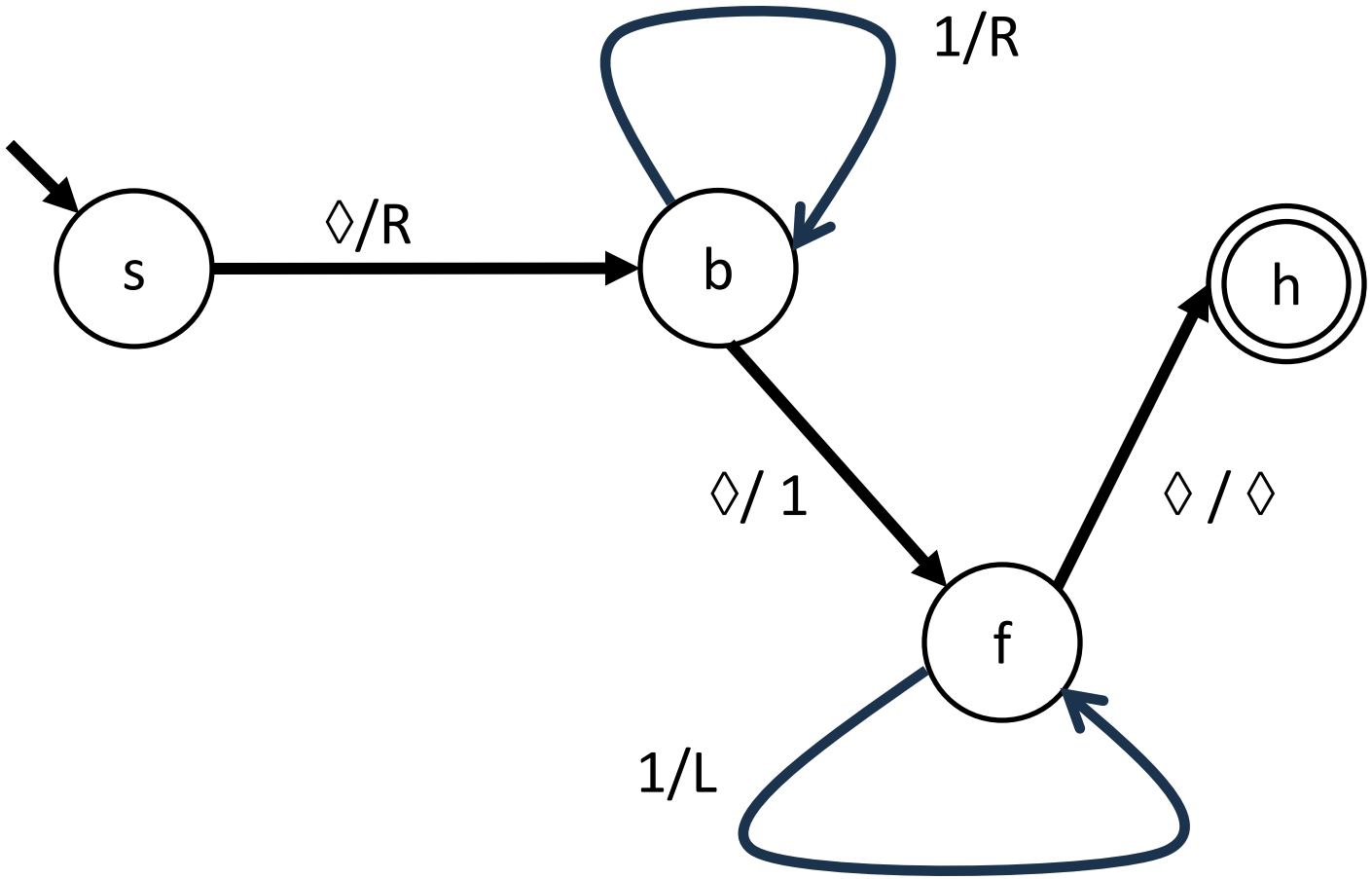
What can be done by computing machines?
Computability

What can realistically be done by computing machines?
Complexity

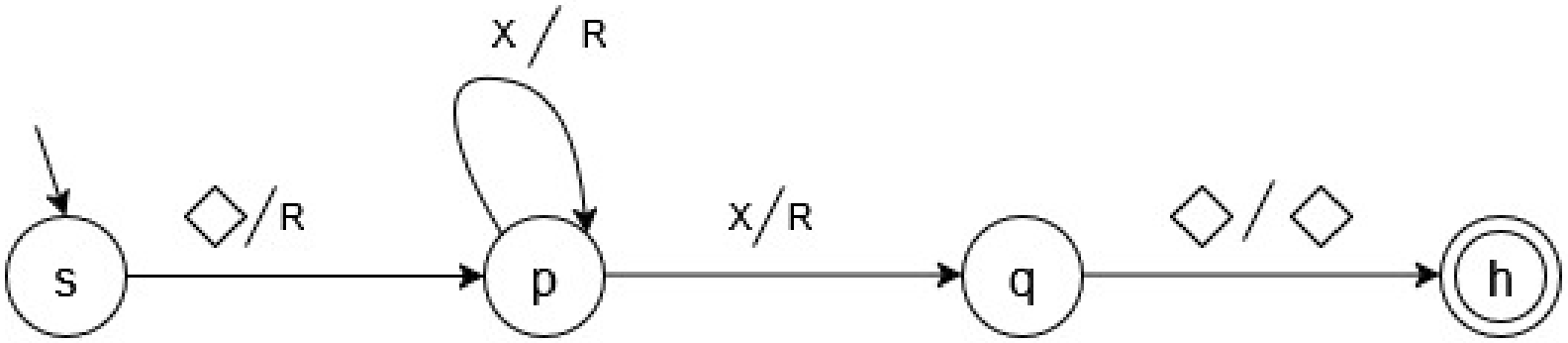
Where on earth to start?
Turing Machines

Exercise: Design a TM to increment unary number

TM to increment unary number & return to where it started



Nondeterministic Turing Machines



A Nondeterministic Turing Machine to accept X^+

It is nondeterministic because there are 2 arcs from state p both labelled with X

State table includes both transitions below !!

Current State	Symbol	Action	Next State
p	X	R	p
p	X	R	q

We Can Prove That ...



Designing a Turing Machine to halt with Y on its tape if a given string **is in a given language**, and halt with N on its tape if it **is not in the language** is not difficult

It's IMPOSSIBLE!

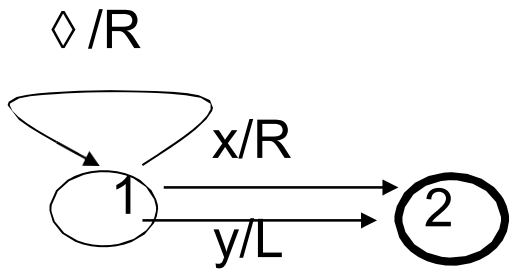
So there are tasks which cannot be solved by any TM

Possible transitions for 2-state TMs with alphabet {◊,x,y}

1. Sl, x -> Sl, **x**
2. Sl, x -> Sl, **y**
3. Sl, x -> Sl, **◊**
4. Sl, x -> Sl, **L**
5. Sl, x -> Sl, **R**
6. Sl, **y** -> Sl, x
7. Sl, **y** -> Sl, y
8. Sl, **y** -> Sl, ◊
9. Sl, **y** -> Sl, L
10. Sl, **y** -> Sl, R
11. Sl, ◊ -> Sl, x
12. Sl, ◊ -> Sl, y
13. Sl, ◊ -> Sl, ◊
14. Sl, ◊ -> Sl, L
15. Sl, ◊ -> Sl, R

And repeat all 15 above with S_{Halt} being the output state.

TMs as Turing numbers



Let ◊ = 000
Let x = 001
Let y = 010
Let LEFT = 011
Let RIGHT = 100
Let S1 = 101
Let S2 = 110

Represent each transition
(State, Symbol) -> (State, Symbol) as:
State Symbol State Symbol

101 000 101 100
101 001 110 100
101 010 110 011

Can list every Deterministic Turing Machine

- Assign a unique binary number to LEFT, RIGHT and every symbol and state
- Given a machine **with N states**, it can have only one of a **finite possible number of state tables**.
- Systematically list all the possible transitions for a 2 state machine, then a 3 state machine, etc

Therefore, there are **countably infinite** Turing Machines.

Countable infinity

Any infinite set that can be put in one to one correspondence with the natural numbers is countably infinite.

Natural numbers:	0, 1, 2, 3, 4, 5, 6, 7, 8 …
Even numbers:	2, 4, 6, 8, …
Prime numbers:	2, 3, 5, 7, …
Integers:	0, 1, 1, 2, 2, 3, 3, 4, 4 …

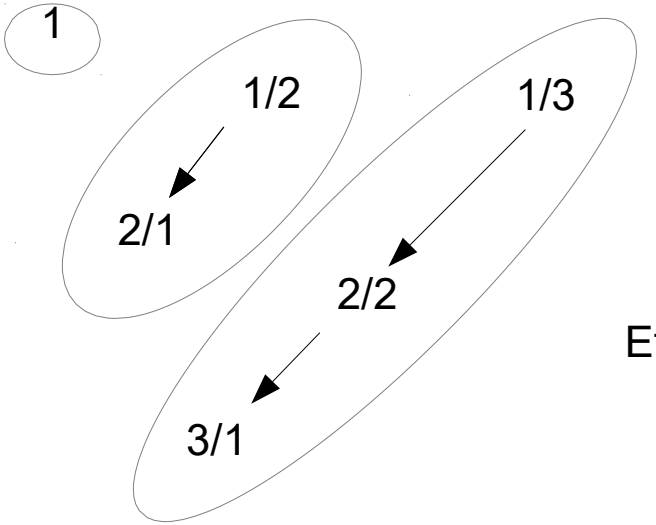
Rational numbers are countable

1/1	1/2	1/3	1/4	⋯
2/1	2/2	2/3	2/4	⋯
3/1	3/2	3/3	3/4	⋯
4/1	4/2	4/3	4/4	⋯
c	c	c	c	⋯

Rational numbers are countable

1/1	1/2	1/3	1/4	⋯
2/1	2/2	2/3	2/4	⋯
3/1	3/2	3/3	3/4	⋯
4/1	4/2	4/3	4/4	⋯
c	c	c	c	⋯

Like so along the diagonals:



Etc ...

Uncountably many real numbers

Assume you can set the real numbers out so that you can count them.

For example:

ij

$$
\begin{array}{lccccc}
3. & d_{11} & d_{12} & d_{13} & d_{14} & \cdots \\
3. & d_{21} & d_{22} & d_{23} & d_{24} & \cdots \\
3. & d_{31} & d_{32} & d_{33} & d_{34} & \cdots \\
3. & d_{41} & d_{42} & d_{43} & d_{44} & \cdots \\
c & c & c & c & c & \cdots
\end{array}
$$

Uncountably many real numbers

Create a new real number, J:

for every digit on the diagonal the corresponding digit in J is set to that digit + 1

3.	$d_{11}+1$	d_{12}	d_{13}	d_{14}	...
3.	d_{21}	$d_{22}+1$	d_{23}	d_{24}	...
3.	d_{31}	d_{32}	$d_{33}+1$	d_{34}	...
3.	d_{41}	d_{42}	d_{43}	$d_{44}+1$	...
c	c	c	c	c	...

Uncountably Many Languages

- Given an input alphabet A={0,1}
- There are a **countably infinite** number of strings that can be constructed using this alphabet. {0, 1, 01, 10, 11 ...}
- Call this set of strings: L_{total}
- $P(L_{total})$, the *power set* of L_{total} is the set of all languages using {0,1} i.e. the set of all subsets of L_{total}
- $P(L_{total})$ is **uncountably infinite**

  uncountably many languages can be formed from $P(L_{total})$

Which Means?

- There are languages which no Turing Machine can recognize.
- Therefore there are problems they cannot compute.

Powerset : uncountably infinite

	e1	e2	e3	e4	e5	e6	e7	- - -
s1	T	T	T	F	F	F	T	- - -
s2	F	F	F	T	F	F	T	- - -
s3	T	T	F	F	F	F	T	- - -
s4	T	T	T	F	F	F	F	- - -
s5	F	F	F	T	T	F	F	- - -
s6	F	F	F	F	F	F	T	- - -
- - -								

Elements are listed along the top, and subsets are one below the other.

T means that element is in that subset, **F** means that element is not in that subset

Powerset : uncountably infinite

	e1	e2	e3	e4	e5	e6	e7	- - -
s1	T	T	T	F	F	F	T	- - -
s2	F	F	F	T	F	F	T	- - -
s3	T	T	F	F	F	F	T	- - -
s4	T	T	T	F	F	F	F	- - -
s5	F	F	F	T	T	F	F	- - -
s6	F	F	F	F	F	F	T	- - -
- - -								

We can make a subset S that is NOT in this arrangement above, by making S differ from every listed subset along the diagonal. Here, e.g., S = {e2, e3, e4, e6 - - - }

Decidable vs. Acceptable

- A language, **La**, is **acceptable** if:
  - There is a TM which will halt (or say YES) if and only if it is presented with a string in **La**.
- A language, **Ld**, is **decidable** if:
  - It is Acceptable and
  - It will crash or say “No” for every string that is not in **Ld**

- Is every decidable language acceptable? Yes
- Is every acceptable language decidable? No

A Nondecidable Language

Machine	Turing Number
M_0	0100100100111...
M_1	0111000011111...
M_2	1100011110001...
...	
M_i	1010000111111...

- A TM that halts given its own Turing Number, T_i, as input, is called self-terminating
- Otherwise it is not self-terminating.

Halting Problem

Define a language L_h over {0, 1}*:

Set of all Turing numbers, T_i, that halt when passed as input to their own machine: i.e. all T_i such that $M_i(T_i)$ halts.

The halting problem: **Is L_h decidable?**

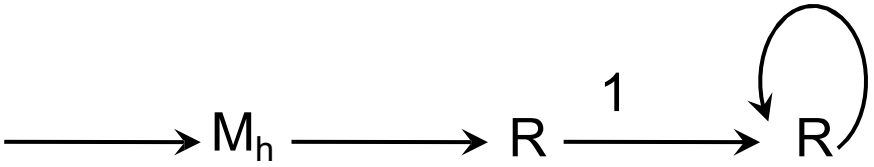
No!

Suppose a machine M_h decides L_h

it outputs a 1 when the answer is YES (will halt) and a 0 when the answer is NO (won't halt).

Halting Problem

What happens if M_0 below is fed its own Turing Number?

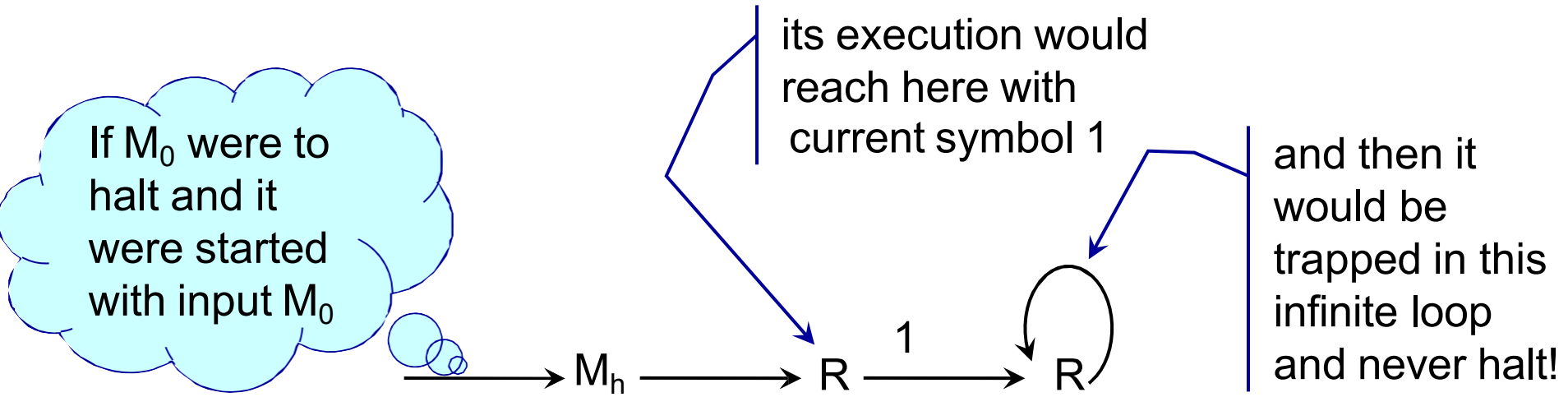


Is M_0 self-terminating or not ?

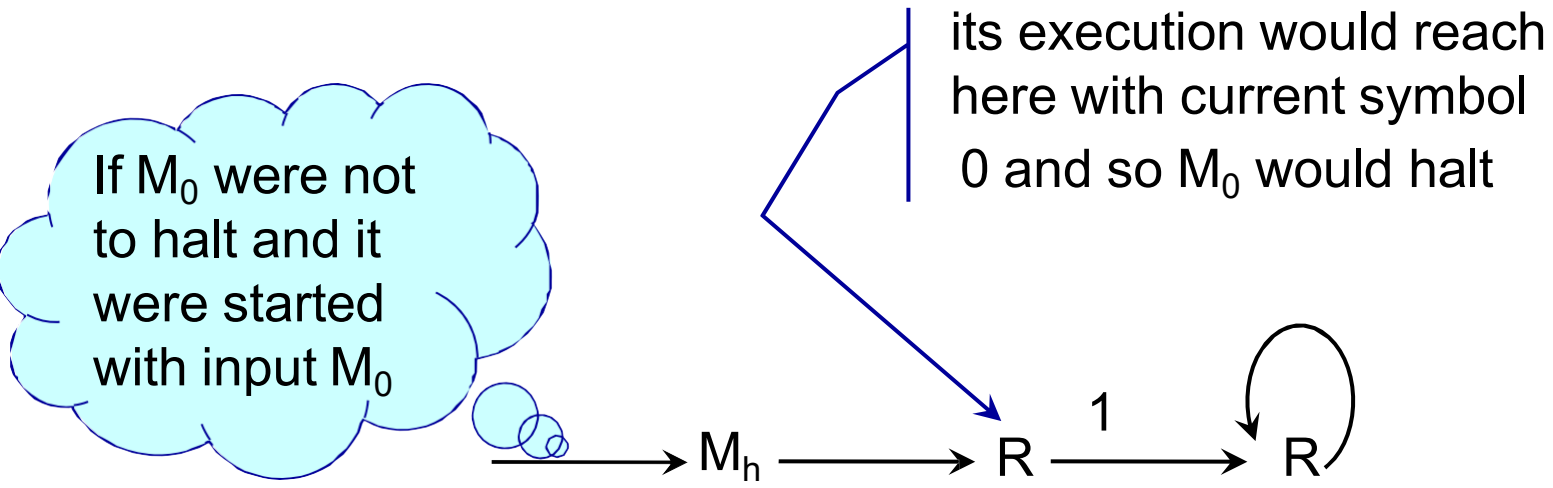
On the next 2 slides, we try each possibility in turn:

1. Yes, M_0 is self-terminating
2. No, M_0 is not self-terminating

Halting Problem



Halting Problem



Halting Problem VI

- M_0 cannot halt or not halt – a contradiction!
- Therefore our assumption is false. L_h is not decidable.

Halting problem: rough idea

Suppose there was a program H that can tell you if ***any*** program would halt (then H outputs Y) or would never halt (then H outputs N) when run with a given input.
H has 2 parameters: the program of interest, and its input: H(p,i)

Well, suppose I write a program P that calls H, and then just goes into an infinite loop if the output of H is Y.
P also has 2 parameters: the program of interest, and its input: P(p,i)

No problem at all!

Except, what would happen if we gave P as input to itself (every program is assembled into a binary representation): P (P,P) ????

Any execution like P(P,P) either halts or it does not halt.
But ***neither*** is the case for P(P,P). So H cannot exist.

Halting problem: idea behind it

1. Every TM can be represented as a binary number
2. Every TM *input* can be represented as a binary number
3. If a TM, say H, can solve the halting problem, it needs 2 inputs: the binary number defining the TM, and the binary number representing its input. It must then write ◊1 if that machine halts on that input, or write 0 if it does not halt on that input.
4. We show by contradiction that this is not possible, by creating a TM, say W, that extends H as follows: if H answered 1 go into an infinite loop (i.e. if H says any execution would halt, W goes into an infinite loop) and if H answered ◊0 then halt (i.e. if H says any execution would not halt, W stops)
5. What does H output if you ask it: Does W halt when given W as input?
6. If H answers 1 (yes) it's wrong. If H answers 0 (no) it's wrong

Another way to see the halting problem

Jo says his program H(P,D) can take any program P and input D and say Y if P(D) or say N if P(D) does not halt. Show him Opp:

```
Opp(P) // P is a program
{       X = H(P,P);
        IF (X == 'N') { PRINT ("STOPPING!"); EXIT( ) }
        // H did not answer N, it answered Y because H(P,P) halts
        Y = 8;
        WHILE (Y > 0) { Y++ }
}
```

Does Opp(Opp) halt? Assume Opp(Opp) halts.No, contradiction!

Assume Opp(Opp) does not halt. Oops, also contradiction!

Yet another way to see the Halting Problem

- Assume we have algorithm A with inputs:
  - P, a program
  - I, an input for that program. Then
    - A(P, I) = 1 if P halts on input I
    - A(P, I) = 0 if P does not halt on input I
- Define Q(P):
  - Q(P) halts if A(P, P) = 0
    - i.e. Q halts if program P does not halt with itself as input
  - Q(P) does not halt if A(P, P) = 1
    - i.e. Q does not halt if program P halts with itself as input
- Substitute Q for P
  - Q(Q) halts if A(Q, Q) = 0
    - i.e. if program Q does not halt on input Q
  - Q(Q) does not halt if A(Q, Q) = 1,
    - i.e. if program Q halts on input Q
- Contradiction. Q halts when it doesn't halt and vice-versa.

Church-Turing Thesis

- A Turing machine can compute any computable function.
  - Or: any computable function is Turing computable
  - No machine with greater computational power can exist
  - Not proven — generally accepted